Technical Assignment: Production-Ready RAG-Based Healthcare Knowledge Assistant

Position: Technical Lead Backend

Duration: 5-7 days

Submission Format: GitHub repository with complete code, documentation, and deployment guide

---

1. Executive Summary

You are tasked with designing, implementing, and deploying a production-ready **Retrieval-Augmented Generation (RAG)** system that serves as an internal company ChatGPT for healthcare domain. This system must seamlessly integrate medical knowledge bases, provide accurate clinical decision support, and operate at scale with enterprise-grade reliability, security, and compliance.

This assignment evaluates your expertise in: **Gen AI/LLM architecture, RAG patterns, system design, backend engineering, cloud deployment, database design, and technical leadership mindset.**

---

2. Problem Statement

Your healthcare organization needs an intelligent knowledge assistant that:

- **Retrieves accurate medical information** from multiple sources (clinical guidelines, research papers, company policies, HL7 standards) without hallucination
- **Provides real-time context-aware responses** to clinical and operational queries
- **Scales to 500+ concurrent users** with sub-2-second response times
- **Maintains HIPAA compliance** and data privacy
- **Supports audit trails** for regulatory compliance
- **Handles medical terminology** and complex clinical reasoning
- **Updates knowledge base dynamically** without system downtime

---

3. Core Requirements

3.1 Architecture & System Design

- **Design a microservices-based RAG architecture** with clear separation of concerns (ingestion, retrieval, LLM, API, monitoring)
- **Propose database strategy:**
  - Vector database (Pinecone, Weaviate, Milvus, or PostgreSQL pgvector) for embeddings
  - SQL database (PostgreSQL) for metadata, users, queries, audit logs
  - NoSQL (Redis) for caching, session management, rate limiting
- **Design for distributed systems:** Handle failures, eventual consistency, and recovery
- **Include observability:** Logging, monitoring (Prometheus/Datadog), tracing, alerting

3.2 Implementation Requirements

- **Backend Framework:** Python (FastAPI) or Node.js (Express/Fastify) or Java (Spring Boot)
  - Must include proper error handling, validation, and security
- **LLM & RAG Stack:**
  - Use LangChain or LlamaIndex for RAG pipeline
  - Support both proprietary LLMs (GPT-4, Claude) and open-source models (Llama 2, Mistral)
  - Implement proper prompt engineering and few-shot learning
- **Vector Database Integration:**
  - Implement semantic search with similarity scoring
  - Support hybrid search (keyword + semantic)
  - Batch embedding with chunking strategy
- **APIs to Build:**
  - `POST /api/v1/knowledge/ask` — submit query, get response with source citations
  - `POST /api/v1/knowledge/ingest` — upload documents for RAG indexing
  - `GET /api/v1/knowledge/history` — retrieve query history with context
  - `GET /api/v1/health` — system health and component status
  - `POST /api/v1/admin/reindex` — rebuild vector indices
  - `POST /api/v1/knowledge/ask` — submit query, get response with source citations
  - `POST /api/v1/knowledge/ingest` — upload documents for RAG indexing
  - `GET /api/v1/knowledge/history` — retrieve query history with context
  - `GET /api/v1/health` — system health and component status
  - `POST /api/v1/admin/reindex` — rebuild vector indices

- **Authentication & Authorization:**
  - JWT-based API authentication
  - Role-based access control (RBAC) for sensitive medical data
  - Audit logging for all queries and document access

3.3 Data & Knowledge Management

- **Document Ingestion:**
  - Support multiple formats: PDF, DOCX, TXT, medical EHR exports
  - Implement chunking strategy (overlapping chunks for context retention)
  - Version control for knowledge base updates
- **Embedding Strategy:**
  - Use domain-specific embeddings (BioGPT, SciBERT, or domain-finetuned models)
  - Implement batch processing for large document sets
  - Cache embeddings for frequently accessed content
- **Query Enhancement:**
  - Query expansion (synonyms, abbreviations common in healthcare)
  - Multi-hop reasoning (connecting related medical concepts)
  - Confidence scoring for retrieved results

3.4 Compliance & Security

- **HIPAA Compliance:**
  - Data encryption at rest (AES-256) and in transit (TLS 1.3)
  - Access control and role-based permissions
  - Audit logs for all data access with timestamps
- **Data Privacy:**
  - PII detection and masking in responses
  - Query sanitization to prevent prompt injection
  - Rate limiting and abuse prevention
- **Security:**
  - API authentication (JWT/OAuth2)
  - Input validation and sanitization
  - CORS configuration for frontend integration

3.5 Deployment & DevOps

- **Containerization:** Docker with optimized images
- **Orchestration:** Kubernetes manifests (Deployment, Service, StatefulSet, ConfigMap, Secrets)
- **Cloud Deployment:** AWS (ECS/EKS), GCP (GKE), or Azure (AKS)
  - Include auto-scaling policies
  - Load balancing strategy
  - Database backup and recovery

- **CI/CD Pipeline:** GitHub Actions, Jenkins, or GitLab CI
  - Automated testing (unit, integration, performance)
  - Deployment automation with rollback capability

3.6 Testing & Performance

- **Unit Tests:** >80% code coverage
- **Integration Tests:** RAG pipeline end-to-end
- **Performance Tests:**
  - Query latency: <2 seconds (p95)
  - Throughput: Support 500+ concurrent users
  - Embedding generation: Batch process 1000+ documents/minute
- **Reliability:** 99.9% uptime SLA (include redundancy plan)

---

4. Deliverables

Phase 1: System Design (Day 1-2)

- **Architecture Diagram:**
  - System components (API Gateway, LLM Service, Vector DB, SQL DB, Cache, Message Queue)
  - Data flow from document ingestion to user query response
  - Deployment topology (Kubernetes/Cloud)
- **Design Document (5-10 pages):**
  - High-level architecture and component interactions
  - Database schema (ERD for SQL, index design for vector DB)
  - API specification (OpenAPI/Swagger)
  - Security & compliance strategy
  - Scalability and performance considerations
  - Failure scenarios and recovery strategy
- **Tech Stack Decision:**
  - Justify choice of LLM (proprietary vs. open-source)
  - Vector DB selection criteria
  - Backend framework reasoning
  - Cloud platform choice

Phase 2: Core Implementation (Day 3-5)

- **Backend API:**
  - Fully functional FastAPI/Express backend with all endpoints
  - Request validation, error handling, logging
  - Database models and migrations
- **RAG Pipeline:**
  - Document ingestion and chunking logic
  - Embedding generation (batch and real-time)
  - Vector similarity search implementation
  - LLM integration with prompt templates
  - Response generation with source citations
- **Database Setup:**
  - PostgreSQL schema with audit logging
  - Vector DB index setup and query optimization
  - Redis caching strategy
- **Authentication & Authorization:**
  - JWT token generation and validation
  - RBAC middleware implementation
  - Audit logging for sensitive operations
- **Code Quality:**
  - Clean code architecture (separation of concerns)
  - Comprehensive error handling
  - Logging and monitoring hooks
  - Type hints (Python) or TypeScript

Phase 3: Deployment & DevOps (Day 6)

- **Containerization:**
  - Production-ready Dockerfiles for backend, embedding service, cache
  - Docker-compose for local development
- **Kubernetes Manifests:**
  - Deployment manifests for all services
  - StatefulSet for databases (PostgreSQL, Vector DB)
  - ConfigMap for configuration, Secrets for credentials
  - Service and Ingress definitions
  - HorizontalPodAutoscaler for auto-scaling
  - Health checks (liveness & readiness probes)
- **Monitoring & Observability:**
  - Prometheus metrics collection
  - Structured logging (JSON format)
  - Distributed tracing (Jaeger/OpenTelemetry)
  - Alerting rules for key SLIs

- **CI/CD Pipeline:**
  - Automated tests on every commit
  - Container image building and registry push
  - Automated deployment to staging/production
  - Rollback strategy

Phase 4: Testing & Documentation (Day 7)

- **Testing Suite:**
  - Unit tests for each component (embedding, retrieval, API)
  - Integration tests for RAG pipeline
  - Load testing with 500+ concurrent users
  - Security testing (SQL injection, prompt injection)
- **Documentation:**
  - README with setup instructions
  - Architecture decision records (ADRs)
  - API documentation (Swagger/Postman collection)
  - Deployment guide (step-by-step for AWS/GCP/Azure)
  - Operational runbook (troubleshooting, scaling, backups)
  - RAG pipeline tuning guide (for future engineers)
- **Performance Report:**
  - Latency benchmarks (query response time, embedding generation)
  - Throughput results (QPS, concurrent users)
  - Resource consumption (CPU, memory, network)
  - Optimization recommendations

---

5. Evaluation Criteria

Criterion	Weight	Excellent (9-10)	Good (7-8)	Acceptable (5-6)
System Design & Architecture	20%	Comprehensive, scalable design with clear rationale; handles edge cases; considers failure modes	Solid design with most components; minor gaps in error handling	Basic design covers core requirements but lacks depth in optimization
RAG Implementation Quality	25%	Fully functional RAG pipeline; proper chunking, embedding, retrieval; excellent source attribution	Working RAG pipeline; minor issues in retrieval accuracy or performance	RAG works but lacks optimization; basic retrieval
Backend Code Quality	20%	Clean, modular code; comprehensive error handling; logging; >80% test coverage	Good code organization; decent error handling; ~70% test coverage	Functional code; minimal error handling; <60% test coverage
DevOps & Deployment	15%	Production-ready Docker/K8s; CI/CD pipeline; monitoring; scaling strategy	Docker/K8s working; basic CI/CD; partial monitoring	Container setup functional; limited deployment automation
Documentation & Communication	10%	Comprehensive docs, architecture diagrams, API specs, deployment guide	Good documentation; some diagrams; deployment steps included	Basic documentation; minimal diagrams
Security & Compliance	10%	HIPAA considerations, encryption, audit logs, access control, PII protection	Basic security measures; some compliance considerations	Minimal security; limited compliance thinking

---

6. Technical Specifications

6.1 Technology Stack (Suggested)

- **Backend:** Python FastAPI or Node.js/TypeScript (Express)
- **RAG Framework:** LangChain or LlamaIndex
- **LLM:** OpenAI GPT-4 / Claude 3 (or open-source: Llama 2, Mistral 7B)
- **Vector DB:** Pinecone, Weaviate, or PostgreSQL pgvector
- **SQL DB:** PostgreSQL
- **Cache:** Redis
- **Embedding Model:** OpenAI ada-002 or domain-specific (BioGPT, SciBERT)
- **Container:** Docker
- **Orchestration:** Kubernetes

- **Cloud:** AWS (Lambda, RDS, ECS/EKS) / GCP (Cloud Functions, Cloud SQL, GKE) / Azure
- **Monitoring:** Prometheus, Grafana, ELK Stack, Datadog
- **CI/CD:** GitHub Actions, GitLab CI, or Jenkins

6.2 Performance Targets

Metric	Target
Query Response Time (p95)	<2 seconds
Embedding Generation (per 1000 docs)	<60 seconds
Concurrent Users	500+
System Uptime (SLA)	99.9%
Code Test Coverage	>80%

---

7. Sample Knowledge Base Content

For demonstration, include sample healthcare documents:

- Clinical practice guidelines (e.g., diabetes management, cardiology protocols)
- Hospital policies and procedures
- Medical terminology glossary
- HL7/FHIR standards reference
- Medication interaction database
- Case study examples (de-identified)

Note: Use publicly available, de-identified, or synthetic healthcare data for development.

---

8. Submission Instructions

1. **GitHub Repository Structure:**

```
rag-healthcare-assistant/
├── backend/                  # FastAPI/Express application
│   ├── app/                 # Main application code
│   ├── tests/               # Unit & integration tests
│   ├── Dockerfile           # Container definition
│   └── requirements.txt      # Dependencies
├── rag/                     # RAG pipeline module
│   ├── ingestion.py         # Document processing
│   ├── retrieval.py         # Vector search
│   ├── generation.py        # LLM prompting
│   └── tests/               # RAG tests
├── kubernetes/              # K8s manifests
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   └── secrets-example.yaml
├── docker-compose.yml       # Local development stack
├── docs/                    # Documentation
│   ├── ARCHITECTURE.md       # System design
│   ├── API.md               # API documentation
│   ├── DEPLOYMENT.md        # Deployment guide
│   ├── RUNBOOK.md           # Operational guide
│   └── DECISIONS.md         # Architecture decisions
├── .github/workflows/       # CI/CD pipeline
├── README.md                # Quick start guide
└── .env.example             # Environment variables template
```

2. **Submission Package:**
   - GitHub repository link (public or provide access)
   - README with setup instructions
   - Summary document (1 page) with highlights and design decisions
3. **How to Run Locally:**
   - Clear instructions for docker-compose setup
   - Test data and sample queries included
   - Should be runnable in <10 minutes

---

9. Interview Discussion Topics

After submission, be prepared to discuss:

- Why you chose specific technologies (LLM, vector DB, backend framework)
- Tradeoffs in your design (latency vs. accuracy, cost vs. performance)
- How you ensured HIPAA compliance and data security
- Scaling strategy (how would you scale to 1M+ queries/day?)
- Monitoring and alerting approach (how would you detect issues?)
- RAG limitations and how you mitigated hallucinations
- Testing strategy and how you ensured reliability
- Technical challenges encountered and solutions
- If you were to optimize further, what would you do?

---

10. Success Criteria

Your submission will be considered successful if:

- ✅ Complete, production-ready RAG system deployed and working
- ✅ System design document with clear architecture and rationale
- ✅ All APIs functional with proper authentication and error handling
- ✅ Vector DB integration with semantic search working smoothly
- ✅ Clean, well-tested code with >80% coverage
- ✅ Docker/Kubernetes manifests ready for deployment
- ✅ Comprehensive documentation (API, deployment, runbook)
- ✅ Performance benchmarks meeting targets (query latency <2s, 500+ concurrent users)
- ✅ Security & compliance considerations addressed (HIPAA, PII, audit logs)
- ✅ Clear operational insights (monitoring, logging, alerting)

This is a comprehensive full-stack assignment that demonstrates technical leadership, Gen AI expertise, system design maturity, and production engineering skills.